Problem 1: Understand matrix memory in PyTorch

GStar Bootcamp

Abstract

Derive causal matrix memory, then implement sequential and chunked linear attention in PyTorch.

Starter file: `problem_1.py`. **Checker:** `python autograder.py -p1 -device cpu`. **Reading path:** Overview. Next: P2.

Mathematical background: what changes after softmax?

Start with the operator from Assignment 1. A causal softmax-attention head forms a weighted average of visible values [1]:

$$\boldsymbol{o}_t^{\text{soft}} = \sum_{j \le t} a_{tj} \boldsymbol{v}_j, \qquad a_{tj} = \frac{\exp(\boldsymbol{q}_t^\top \boldsymbol{k}_j)}{\sum_{u \le t} \exp(\boldsymbol{q}_t^\top \boldsymbol{k}_u)}.$$

The weights are nonnegative and sum to one. Changing one visible score changes its denominator and therefore generally changes every weight for that query. In matrix notation,

$$\mathbf{O}^{\text{soft}} = \text{softmax}_{\text{row}}(\mathbf{Q}\mathbf{K}^\top + \mathbf{B})\mathbf{V}, \qquad B_{tj} = \begin{cases} 0, & j \le t, \\ -\infty, & j > t. \end{cases}$$

The softmax mask is *additive*. Setting a forbidden logit to zero would give it a nonzero exponential and would not remove it from the average.

For this assignment, change the weighting rule to an unnormalized dot product:

$$\boldsymbol{o}_t^{\text{lin}} = \sum_{j \le t} (\boldsymbol{q}_t^\top \boldsymbol{k}_j) \boldsymbol{v}_j, \qquad \mathbf{O}^{\text{lin}} = \text{tril}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}.$$

There is no exponential or query-dependent denominator. An invalid *linear score* must be zero because it is multiplied directly by a value. Valid weights may be negative and need not sum to one; the output need not lie in the convex hull of the values. If two visible unit scalar keys and the query are all 1, with values 1 and 3, softmax returns 2 while this linear operator returns 4. They are different attention mechanisms, even when supplied with identical inputs.

Why the new rule can become a recurrent memory

With zero initial memory, regroup the finite sum:

$$\boldsymbol{o}_t^{\text{lin}} = \left(\sum_{j \le t} \boldsymbol{k}_j \boldsymbol{v}_j^\top \right)^\top \boldsymbol{q}_t = \mathbf{S}_t^\top \boldsymbol{q}_t.$$

The sum defining $\mathbf{S}_t$ is independent of the current query. Once stored, it can be updated with one new outer product instead of retaining a new key/value pair. For nonzero initial memory, add $\mathbf{S}_0$ to the sum. This gives the recurrence below. The parallel score expression, token recurrence and chunk

form compute *the same linear operator* in exact arithmetic. None is a rearrangement that exactly reproduces softmax attention.

Without the causal mask, matrix associativity gives $(\mathbf{Q}\mathbf{K}^\top)\mathbf{V} = \mathbf{Q}(\mathbf{K}^\top\mathbf{V})$. With causality, one full-sequence $\mathbf{K}^\top\mathbf{V}$ would leak future tokens. Instead, each query reads the prefix state $\mathbf{S}_t$; chunking separates the earlier-prefix state from a local triangular product. Softmax cannot use this particular reassociation because its nonlinear score transformation sits between the two matrix products.

For fixed key/value widths, the recurrent arithmetic costs $O(TD_kD_v)$ and its carried memory has D_kD_v entries. A dense attention head instead performs pairwise score/value work over $O(T^2)$ token pairs. This comparison concerns attention arithmetic and recurrent history, not all activations or the output tensor. P1 first establishes the identity; P2 and P3 choose different GPU execution schedules for it.

How this relates to normalized linear attention

Another common definition uses a separable feature-map kernel [2]. For a fixed-width map $\boldsymbol{\phi}$ and a positive denominator, let

$$\mathbf{H}_t = \sum_{j\le t} \boldsymbol{\phi}(\boldsymbol{k}_j)\boldsymbol{v}_j^\top, \quad \boldsymbol{z}_t = \sum_{j\le t} \boldsymbol{\phi}(\boldsymbol{k}_j), \quad \boldsymbol{o}_t = \frac{\mathbf{H}_t^\top \boldsymbol{\phi}(\boldsymbol{q}_t)}{\boldsymbol{z}_t^\top \boldsymbol{\phi}(\boldsymbol{q}_t)}.$$

This form needs an additional normalizer state $\boldsymbol{z}_t$. The starter uses the identity-feature, *unnormalized* dot-product operator above: do not add that denominator or a feature-map activation.

Reading the suggested talk with this assignment

The opening attention-to-memory discussion in Yang's talk is useful companion reading [3]. Its memory is $\widetilde{\mathbf{S}}_t = \sum_j \boldsymbol{v}_j\boldsymbol{k}_j^\top$, whereas ours is $\mathbf{S}_t = \widetilde{\mathbf{S}}_t^\top$. Thus its read $\widetilde{\mathbf{S}}_t\boldsymbol{q}_t$ becomes $\mathbf{S}_t^\top\boldsymbol{q}_t$, and its chunk write $\mathbf{V}^\top\mathbf{K}$ becomes $\mathbf{K}^\top\mathbf{V}$. Keep this orientation when translating equations to kernels.

Before coding. For a fixed query, explain why the linear rule can read a prefix matrix, why softmax needs its query-dependent normalizer, and why a full-sequence matrix would violate causality.

Background: from a sum to a recurrence

For causal linear attention, define

$$\boldsymbol{o}_t = \sum_{u=1}^{t} (\boldsymbol{q}_t^\top \boldsymbol{k}_u)\boldsymbol{v}_u. \tag{1.1}$$

Associativity allows the query to read a matrix containing all key–value outer products:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \boldsymbol{k}_t\boldsymbol{v}_t^\top, \qquad \boldsymbol{o}_t = \mathbf{S}_t^\top\boldsymbol{q}_t. \tag{1.2}$$

With nonzero $\mathbf{S}_0$, Eq. (1.1) also contains $\mathbf{S}_0^\top\boldsymbol{q}_t$. The state has D_kD_v elements regardless of sequence length. The output still depends on every past update, but the recurrence summarizes them in this fixed matrix.

Worked example

Use $D_k = 2$, $D_v = 1$, $\mathbf{S}_0 = (0,0)^\top$. Let $\boldsymbol{k}_1 = (1,0)^\top$, $v_1 = 2$, $\boldsymbol{k}_2 = (0,1)^\top$, $v_2 = 3$, and $\boldsymbol{q}_2 = (1,1)^\top$. Then

$$\mathbf{S}_1 = \begin{bmatrix}2\\0\end{bmatrix}, \quad \mathbf{S}_2 = \begin{bmatrix}2\\3\end{bmatrix}, \quad o_2 = \begin{bmatrix}2 & 3\end{bmatrix}\begin{bmatrix}1\\1\end{bmatrix} = 5.$$

Reading $\mathbf{S}_1$ at token 2 would incorrectly omit the current value and return 2.

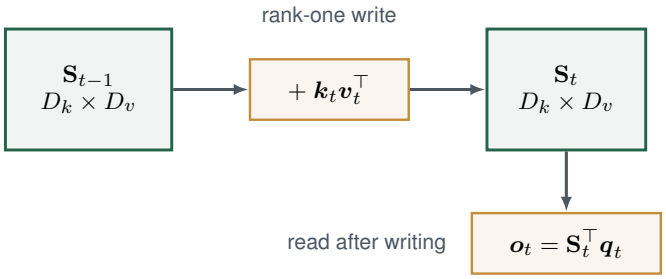


Figure 1. One token updates a matrix memory and then reads it. The key chooses state rows; the value supplies the write across value channels.

Your task

Complete `linear_reference_student` and `linear_chunk_student` in `problem_1.py`. First establish the sequential definition; then reproduce it with chunk matrix products. The equations omit query scaling. In the starter, the optional `scale` is already applied to `q` before your implementation regions; use `scale=1.0` to reproduce the worked examples.

Problem 1: derive and implement the chunk form

A chunk splits old and new information

For local row r, expand Eq. (1.2) within a chunk:

$$\mathbf{S}_{cC+r} = \mathbf{S}_{\text{in}} + \sum_{u=1}^{r} \boldsymbol{k}_u \boldsymbol{v}_u^\top.$$

Stacking the output rows gives

$$\mathbf{O} = \Big[\underbrace{\mathbf{Q}\mathbf{S}_{\text{in}}}_{\text{earlier chunks}} + \underbrace{\text{tril}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}}_{\text{current chunk}} \Big], \tag{1.3}$$

$$\mathbf{S}_{\text{out}} = \mathbf{S}_{\text{in}} + \mathbf{K}^\top \mathbf{V}. \tag{1.4}$$

The $C \times C$ score tile is bounded by the chunk size. The complete sequence's $T \times T$ score matrix is unnecessary.

Step-by-step implementation

1. **Sequential update and read (25 points).** At each token, form the outer product, add it to the state, and read the updated state. Preserve a nonzero initial state.
2. **Chunk update and output (35 points).** Compute an inclusive lower mask, use Eq. (1.3) with the entry state, and only then advance the carried state using Eq. (1.4).
3. **Explain the diagonal and state timing (15 points).** In your solution comments, explain why the local score tile includes its diagonal and why output must use the state entering the chunk.

```
# Sequential skeleton: allocation and token indexing are provided.
for t in range(seq_len):
    qt, kt, vt = q[:, :, t], k[:, :, t], v[:, :, t]
    # TODO P1.A: update the [B, H, Dk, Dv] state.
    # TODO P1.B: append the read of the updated state.

# Chunk skeleton: Q/K/V are row-stacked chunk tensors.
# TODO P1.C: form an inclusive causal score tile.
# TODO P1.D: combine entry-state and local contributions.
# TODO P1.E: advance the state to the chunk boundary.
```

Verification

Use the worked example, then test random tensors with $T \in \{1, 63, 64, 65, 200\}$ and both zero and nonzero initial state. Compare every output and the final state between your two functions. Before introducing BF16 rounding, perform this comparison in FP64 on small tensors.

Common mistake: reading the exit state

If you substitute $\mathbf{S}_{\text{out}}$ for $\mathbf{S}_{\text{in}}$ in the inter-chunk term, you count the current chunk twice and include future tokens from that chunk. The entry-state term and inclusive local term already account for all valid tokens exactly once.

Problem 1: a complete two-token matrix example

Take $D_k = 2$, $D_v = 3$ and a nonzero initial memory

$$\mathbf{S}_0 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{V} = \begin{bmatrix} 2 & 5 & 1 \\ 7 & 3 & 4 \end{bmatrix}, \quad \mathbf{Q} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}.$$

The first association changes only row 1 of memory; the second changes only row 2:

$$\mathbf{S}_1 = \begin{bmatrix} 3 & 6 & 2 \\ 1 & 1 & 1 \end{bmatrix}, \qquad \mathbf{S}_2 = \begin{bmatrix} 3 & 6 & 2 \\ 8 & 4 & 5 \end{bmatrix}.$$

Reading these states gives $\boldsymbol{o}_1 = (5, 8, 4)^\top$ and $\boldsymbol{o}_2 = (41, 34, 26)^\top$. This case simultaneously tests nonzero initial memory, unequal key/value dimensions, and current-token inclusion.

Recover the same outputs with one chunk

For $C = 2$, decompose output into old memory and local writes:

$$\mathbf{Q}\mathbf{S}_0 = \begin{bmatrix} 3 & 3 & 3 \\ 7 & 7 & 7 \end{bmatrix}, \qquad \operatorname{tril}(\mathbf{Q}\mathbf{K}^\top) = \begin{bmatrix} 1 & 0 \\ 3 & 4 \end{bmatrix}.$$

The local contribution is

$$\begin{bmatrix} 1 & 0 \\ 3 & 4 \end{bmatrix} \mathbf{V} = \begin{bmatrix} 2 & 5 & 1 \\ 34 & 27 & 19 \end{bmatrix}.$$

Adding the two terms reproduces the sequential outputs. The exit memory is $\mathbf{S}_0 + \mathbf{K}^\top \mathbf{V} = \mathbf{S}_2$. Notice that the first local row cannot use the second value, even though the complete chunk is loaded at once.

Why associativity gives exact chunking

The sequential sum groups writes by token. Chunking groups the same writes by their location relative to a query: earlier chunks are summarized in $\mathbf{S}_{\text{in}}$, and current-chunk writes remain explicitly visible through a causal matrix. This is an algebraic regrouping in exact arithmetic; it is not a truncation of the context. Floating-point implementations may differ because they add terms in a different order.

Implementation checklist

1. Identify the accumulation dtype before allocating or reading state.
2. In the sequential function, update memory before the query read.
3. In the chunk function, compute output using the entry memory before advancing it.
4. Permit a short final chunk without requiring T to be divisible by C.
5. Verify $\mathbf{S}_T$ is independent of query values.

For your explanation, derive the first and second output rows above without executing code. Then identify which mistake would remove the old-memory contribution and which mistake would include a future write.

References

[1] A. Vaswani et al. *Attention Is All You Need*. 2017.
https://arxiv.org/abs/1706.03762

[2] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention*. 2020.
https://arxiv.org/abs/2006.16236

[3] S. Yang. *Toward More Expressive yet Scalable RNNs: DeltaNet and Its Variants*. Talk, July 15, 2025.
https://sustcsonglin.github.io/assets/pdf/deltanet_talk_slides.pdf